Lenguaje de Consultas
SQL

Introducción a SQL

- De su traducción al inglés: Structured Query Language (SQL), es un lenguaje declarativo de acceso a bases de datos que combina construcciones del álgebra relacional y del cálculo relacional.
- Originalmente fue desarrollado en los '70 por IBM en su Research Laboratory de San José a partir del cálculo de predicados creado por Codd.
- Hay un compendio de revisiones del estándar SQL definidos por ANSI e ISO, entre ellas:
  - SEQUEL(Structured English QUEry Language), IBM 1976
  - SQL-86 (ANSI SQL), SQL-89 (SQL1),
  - SQL-92 (SQL2), gran revisión del estándar
  - SQL:1999 (SQL3), Añade disparadores, algo de OO, y demás
  - SQL:2003. Añade XML, secuencias y columnas autonuméricas, y más ...
  - SQL: 2005,2008,2011,2016, este último permite búsqueda de patrones, y más ..

Características a SQL

- Idealmente un lenguaje de DB debería permitir al usuario realizar lo siguiente con un mínimo esfuerzo:
  - Crear bases de datos y estructuras de relaciones.
  - Ejecutar tareas de administración de datos: inserción, modificación y borrado de datos desde las relaciones
  - Realizar consultas (queries) simples y complejos
- Como lenguaje, el estándar ISO SQL define dos principales grupos:
  - Data Definition Language (DDL), para definir la estructura de la DB y controlar el acceso a los datos
  - Data Manipulation Language (DML), para recuperar y actualizar datos. Embebido en los lenguajes de programación
- SQL es un lenguaje en donde se especifica qué datos se requiere obtener, en lugar de cómo se obtienen. El cómo se obtienen se refiere al método de acceso.

Data Definition:
Tipos de datos SQL

Tipos de datos escalares SQL

- En lo referente a manipulación de datos los tipos character y bit son referidos como strings. También exact y approximate numeric son referidos como tipos de datos numbers.

- Character consiste de una secuencia de caracteres. Cuando se define una columna de este tipo se debe especificar la longitud: fija o variable. Ejemplo:

```
brachNo CHAR(4)

address VARCHAR(30)
```

TIPO DE DATO	DECLARACIÓN				
boolean	BOOLEAN				
character	CHAR	VARCHAR			
bit*	BIT	BIT VARYNG			
exac numeric	NUMERIC	DECIMAL	INTEGER	SMALLINT	BIGINT
aproximate numeric	FLOAT	REAL	DOUBLE PRECISION		
datetime	DATE	TIME	TIMESTAMP		
interval	INTERVAL				
large objects	CHARACTER LARGE OBJECT			BINARY LARGE OBJECT	

Tipos de datos escalares SQL (2)

- `Exact numeric`: números con una representación exacta.
- Consisten de `precisión` y `escala`: la presición es el número de dígitos que incluye los de la parte entera y decimal. La escala es el número de lugares decimales.
  - `- 1 2 . 3 4 5` Precisión 5 y escala 3
- Hay varios tipos de datos numéricos en esta categoría:
  - `NUMERIC(precision, scale) DECIMAL(precision, scale)`
  - `INTEGER SMALLINT BIGINT`
- ¿Cómo seleccionarlo?: una columna rooms que representa el número de cuartos en una propiedad:

```
rooms SMALLINT
```

Tipos de datos escalares SQL (3)

- Approximate numeric: números con una representación inexacta. La notación es similar a la notación científica, donde la presición es la mantisa
  - FLOAT[precision] REAL DOUBLE PRECISION
- Datetime:
  - DATE, usado para fecha de calendario, con los campos: YEAR, MONTH, DAY
  - TIME, usado para el tiempo, con los campos: HOUR, MINUTE, y SECOND
  - TIMESTAMP, para almacenar DATE y TIME
- Large objects: usados para almacenar archivos de texto largos, o archivos de gráficos.
  - Binary Large Object(BLOB)
  - Character Large Object (CLOB)

Sentencias DDL para definiciones de datos

Creación de bases de datos

- El proceso de crear una DB, comunmente tiene particularidades que dependen el producto de software.
- El estándar ISO define, de forma general, a una DB como parte de un entorno, y solo provee de sentencias para la definición de schemas.
- Un `schema` tiene un `propietario` y es una colección de objetos de una base de datos que están de cierta forma relacionados por el nombre del schema.
- Los objetos en el esquema pueden ser: tablas, vistas, entre otros.
  - `CREATE SCHEMA SqlTests AUTHORIZATION dbtest;`
- Un esquema puede ser borrado usando la sentencia:
  - `DROP SCHEMA SqlTets;`

Creación de Tablas

- Esta tarea se consigue usando la sentencia `CREATE TABLE`, la siguiente sentencia crea una nueva relación de nombre `department` con 3 atributos:

```
CREATE TABLE department
( dept_name VARCHAR(200),
  building VARCHAR(15),
  budget  NUMERIC(12, 2),
  PRIMARY KEY (dept_name));
```

- Se divide en 2 secciones: (1) la definición de columnas, y (2) las restricciones de integridad.
- Para remover una relación se utiliza la sentencia `DROP TABLE t;`

Restricciones de clave

- La especificación de `primary key({L})` : define la lista L de atributos de clave primaria.

```
CREATE TABLE course
(course_id VARCHAR(7),
 title VARCHAR(50) NOT NULL,
 dept_name VARCHAR(20),
 credits NUMERIC(2,0),
 PRIMARY KEY (course_id),
 FOREIGN KEY (dept_name) REFERENCES department);
```

- La especificación de `foreign key({L}) references s` : define la lista L de atributos de clave foránea y que deben coincidir con algún valor de la clave primaria en `s`.
- En cada tupla de `course`, el valor de `dept_name` debe existir en el atributo de PK de `department`, sin esto, es posible que las tuplas de `course` especifiquen un `dept_name` que no existe.

Cambiar la definición de una Tabla

- El estándar ISO SQL provee la sentencia ALTER TABLE.
- Consiste de seis opciones:
  - añadir (`add`) y eliminar (`drop`) columnas de una tabla
  - añadir y eliminar restricciones de una tabla
  - establecer (`set`) y eliminar (`drop`) el `DEFAULT` para una columna
- Al añadir una nueva columna a una relación existente, son asignados `NULL` a los valores de esta columna
  - `ALTER TABLE r ADD col1 INTEGER;`
  - `ALTER TABLE r DROP col1;`
- Al intentar añadir una restricción a una tabla existente, se verifica que cada fila de la tabla cumpla con la restricción, si alguna no cumple no es posible añadir la restricción

Restricciones de Integridad

- Una restricción de integridad (integrity constraint) asegura que un cambio realizado en la DB no conduzca a la pérdida de la consistencia de datos.
- Ejemplos de restricciones de integridad:
  - El nombre de un `instructor` no puede ser `null`.
  - Dos o más `instructores` no pueden tener el mismo `ID`.
  - Cada nombre de departamento (`dept_name`) en la relación `course` debe tener un valor que coincida en la relación `department`
- Usualmente las restricciones de integridad son parte de la definición de las tablas, sin embargo, pueden ser agregadas mediante la sentencia
  - `ALTER TABLE table-name ADD constraint`

Restricciones solo de tabla

- `not null` : es utilizado columnas obligatorias.
  - `name VARCHAR(20) NOT NULL,`
  - `budget NUMERIC(2,0) NOT NULL`
- `unique ({L})` : define la lista L de atributos de una clave candidata, a diferencia de PK, los atributos declarados pueden ser `nulos`.
- `check (P)` : se especifica un predicado P, es usado para campos que deben satisfacer una condición.
- La claúsula `check(P)` también puede ser utilizada para simular datos enumerados. e.g. La columna semestre debe ser uno de estos valores: 'Fall', 'Winter', 'Spring' o 'Summer'.
- Un claúsula `check`, al igual que una `not null`, pueden aparecer junto a la definición de una columna.

Restricciones de tabla: ejemplo 1

```
create table section
(course_id varchar (8),
 sec_id varchar (8),
 semester varchar (6),
 year numeric (4,0),
 building varchar (15),
 room_number varchar (7),
 time_slot_id varchar (4),
 primary key (course id, sec_id, semester, year),
 check (semester in ('Fall', 'Winter', 'Spring','Summer'))
 foreign key (course_id) references course,
 foreign key (building, room_number) references classroom
);
```

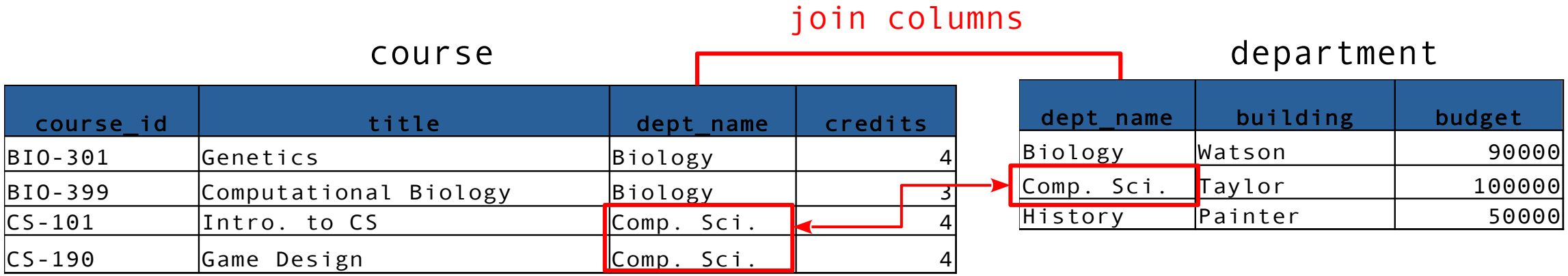
Restricciones de tabla: ejemplo 2

```
create table section
(course_id varchar (8),
 sec_id varchar (8),
 semester varchar (6) check (semester in ('Fall', 'Winter',
                             'Spring','Summer')),
 year numeric (4,0) check (year>1759 and year<2100),
 building varchar (15),
 room_number varchar (7),
 time_slot_id varchar (4),
 primary key (course id, sec_id, semester, year),
 foreign key (course_id) references course,
 foreign key (building, room_number) references classroom
);
```

Integridad Referencial

- Las restricciones de clave foránea son la manera de asegurarnos que los valores de unos atributos coincidan con algún valor de la clave primaria en otra relación.
  - `foreign key(dept_name) references department(dept_name)`
- Puede declararse como parte de la definición de la tabla así:
  - `dept_name varchar(20) references department`
- Cuando una restricción de clave foránea es violada, lo normal es que se rechace la acción. Sin embargo, puede especificarse algo distinto cuando una operación de `delete` o de `update` viola la restricción.
- Considere la siguiente definición de integridad referencial:
  -
```
foreign key(dept_name) references department(dept_name)
                       on update cascade
                       on delete cascade,
```

Violación de Integridad Referencial



course_id	title	dept_name	credits
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to CS	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4

dept_name	building	budget
Biology	Watson	90000
Comp. Sci.	Taylor	100000
History	Painter	50000

- ¿Qué sucede con las tuplas que hacen referencia? cuando se intenta:
  - borrar la tupla con el valor de “Comp. Sci.” en `department`
  - cambiar el valor de `dept_name` de la tupla con valor de “Comp. Sci.” en `department`
- Varias opciones que permiten estas operaciones sin violar la restricción:
  - cascade: es propagar el cambio, e.g., en un operación delete se propaga el borrado
  - set null, o set default: se establece null o el valor default respectivamente

Asignación de nombres a las restricciones

- Las restricciones nombradas son útiles si se desea quitar (drop) la restricción que ha sido previamente definida.
  - ```
    salary numeric (8, 2),
    constraint minsalary check(salary>29000))
    ```
- Luego, será posible quitar la restricción usando su nombre: "minsalary"
  - ```
    alter table instructor drop constraint minsalary;
    ```
- Sin un nombre, el DBMS le asignaría un nombre automático y para poder quitar la restricción es necesario determinar el nombre a través de las opciones provista por el software.